به نام خدا

طراحی کامپیوتری سیستم‌های دیجیتال

پاییز ۱۴۰۵

طراحی شتاب دهنده شبکه عصبی

طراحان: هاتف رضائی - فائزه میثاقی

هدف پروژه :

در این تمرین، هدف طراحی و پیاده‌سازی سخت‌افزار یک شتاب‌دهنده برای **شبکه عصبی کاملاً متصل (Fully Connected Neural Network یا FCNN)** است. در این نوع شبکه عصبی، هر نورون در یک لایه به تمامی نورون‌های لایه بعدی متصل است.

شبکه عصبی موردنظر برای **طبقه‌بندی ارقام دست‌نویس** با استفاده از مجموعه‌داده‌ی **MNIST** به کار گرفته می‌شود. در پیاده‌سازی اصلی شبکه، ورودی‌ها و پارامترهای شبکه، از جمله وزن‌ها و بایاس‌ها، با استفاده از اعداد ممیز شناور **۳۲ بیتی (Floating Point 32 یا FP32)** نمایش داده می‌شوند. همچنین، مقادیر ورودی شبکه از مقادیر پیکسل‌های تصاویر مجموعه‌داده‌ی MNIST تشکیل شده‌اند.

با توجه به هدف این تمرین، که تمرکز آن بر طراحی و پیاده‌سازی سخت‌افزار است، به‌منظور کاهش پیچیدگی محاسبات و ساده‌تر شدن طراحی سخت‌افزاری، در پیاده‌سازی موردنظر از قالب **ممیز شناور ۱۶ بیتی (FP16)** استفاده خواهد شد. تمامی محاسبات ممیز شناور مورد استفاده در شتاب‌دهنده بر اساس قالب **IEEE 754** انجام می‌شوند.

معماری شبکه عصبی:

شبکه عصبی موردنظر از **سه لایه** تشکیل شده است و ساختار آن به صورت زیر تعریف می‌شود:

- **ورودی:** تصویر سیاه‌وسفید با ابعاد 28×28 پیکسل از مجموعه‌داده‌ی MNIST.
- لایه ورودیِ تخت‌شده (Flattened): تصویر ورودی به یک بردار شامل ۷۸۴ مقدار ممیز شناور ۱۶ بیتی (FP16) تبدیل می‌شود.
- **لایه کاملاً متصل اول (Fully Connected):** شامل **۸ نورون** است که هر نورون به تمامی ۷۸۴ ورودی متصل می‌باشد.
- **تابع فعال‌سازی:** پس از محاسبه‌ی لایه‌ی اول، تابع فعال‌سازی **ReLU (Rectified Linear Unit)** بر روی خروجی نورون‌ها اعمال می‌شود.
- **لایه کاملاً متصل دوم:** شامل **۱۰ نورون** است که هر یک از آن‌ها به تمامی ۸ خروجی لایه‌ی قبلی متصل می‌باشند.
- **خروجی:** یک عدد صحیح در بازه‌ی **۰ تا ۹** برای هر کدام از ۱۰ نورون لایه آخر که رقم پیش‌بینی‌شده توسط شبکه را نشان می‌دهد.

بنابراین، مسیر کلی داده در شبکه به صورت زیر است:

28×28→784→8→10

معماری شتابدهنده

این معماری شامل چهار زیرماژول اصلی («SRAM»، «ضرب‌کننده»، «بافر پینگ‌پونگی» و «جمع‌کننده») است که هر کدام در ادامه توضیح داده خواهند شد؛ در پایان نیز نحوه‌ی تولید و اعلام خروجی نهایی شبکه شرح داده می‌شود.

خروجی هر لایه از یک شبکه عصبی کاملا متصل به صورت

$$H_j = \sum_i Wij * xi + C_j$$

در نتیجه برای داشتن یک شتابدهنده نیازمند ماژول های ضرب و جمع (MAC) هستیم. همچنین یک معماری buffer که به صورت پینگ پونگی استفاده می شود در کنار یک ماژول Sram که وزن های این شبکه در آن ذخیره شده است، برای تکمیل مسیر داده نیاز هستند.

SRAM

برای sram نیاز است که رفتار یک ماژول حافظه را شبیه سازی کنید. این ماژول یک آدرس ورودی را گرفته و پس از 7 سیکل یک row (که برای سادگی طراحی در این پروژه، شامل ۱۱۲ بیت است؛ یعنی معادل هفت عدد fp16 شانزده‌بیتی) را خروجی می‌دهد. این اعداد باید در 7 رجیستر نوشته شوند تا بتوان به صورت مداوم از این ماژول استفاده کرد.

ضرب کننده‌ی fp16

این ضرب کننده از یک رجیستر 7 تایی می خواند و در 7 سیکل ضرب هر دو به دوی وزن و ورودی را انجام داده و در هر سیکل هر کدام از نتایج ضرب را تولید و در بافر پینگ پونگی مینویسد. توجه کنید که ورودی و خروجی این ماژول به فرمت fp16 هستند.

بافر پینگ پونگی

از آنجا که ضرب‌کننده در هر سیکل یک مقدار حاصل‌ضرب جدید (به فرمت fp16) تولید می‌کند، اما جمع‌کننده ممکن است برای پردازش این مقادیر به چند سیکل نیاز داشته باشد، از یک بافر دوگانه (پینگ پونگ) استفاده می‌شود تا تولید داده توسط ضرب‌کننده و مصرف آن توسط جمع‌کننده از یکدیگر مستقل شوند. این بافر شامل دو بلوک ثبات است.

در هر لحظه، یکی از این دو بلوک در حالت نوشتن (write) قرار دارد و ضرب‌کننده نتایج جدید خود را در آن ذخیره می‌کند، در حالی که بلوک دیگر در حالت خواندن (read) است و جمع‌کننده مقادیر ذخیره‌شده‌ی دوره‌ی قبلی را از آن می‌خواند و پردازش می‌کند. پس از آنکه نوشتن هفت مقدار در یک بلوک و خواندن کامل بلوک دیگر هر دو به پایان رسیدند، نقش دو بلوک با یکدیگر جابه‌جا می‌شود (پینگ پونگ).

این جابه‌جایی معمولاً با یک بیت انتخاب (select) که وضعیت هر بلوک را مشخص می‌کند پیاده‌سازی می‌شود. برای جلوگیری از رونویسی روی داده‌ی هنوز خوانده‌نشده یا خواندن داده‌ی نامعتبر، لازم است سیگنال‌های وضعیتی مانند full و empty (یا valid/ready) برای هر بلوک در نظر گرفته شوند. با این مکانیزم، ضرب‌کننده می‌تواند بدون توقف (stall) به تولید خروجی ادامه دهد، به شرط آنکه جمع‌کننده با همان نرخ داده‌ها را مصرف کند.

جمع کننده

جمع‌کننده مقادیر حاصل‌ضرب را از سمت خواندنِ بافر پینگ پونگی دریافت کرده و آن‌ها را برای محاسبه‌ی Hj با یکدیگر جمع می‌کند. از آنجا که تعداد ورودی‌های هر نورون (۷۸۴ در لایه‌ی اول و ۸ در لایه‌ی دوم) بسیار بیشتر از ظرفیت هفت‌تایی بافر است، این جمع باید به‌صورت انباشتی (accumulation) و در طول چندین دوره‌ی پر شدن بافر انجام شود.

به این منظور، جمع‌کننده یک رجیستر انباشت‌گر (accumulator) داخلی دارد که مقدار جمع جزئیِ هر نورون را نگه می‌دارد. در هر دوره، هفت مقدار خوانده‌شده از بافر با یکدیگر و سپس با مقدار پیشینِ انباشت‌گر جمع می‌شوند و نتیجه دوباره در انباشت‌گر ذخیره می‌شود. این عملیات تا زمانی که تمامی ورودی‌های مربوط به یک نورون پردازش شوند، تکرار می‌شود.

پس از پردازش آخرین ورودیِ هر نورون، مقدار بایاس (Cj) به انباشت‌گر اضافه شده و مقدار نهاییِ Hj تولید می‌شود. این مقدار در لایه‌ی اول به واحد فعال‌سازیِ ReLU ارسال می‌شود و در لایه‌ی دوم مستقیماً به‌عنوان خروجیِ خامِ آن نورون در نظر گرفته می‌شود که پس از مقایسه با سایر نورون‌های لایه‌ی آخر، رقمِ پیش‌بینی‌شده را مشخص می‌کند.

جمع‌کننده باید از یک هسته‌ی جمعِ fp16 مطابق با استانداردِ IEEE 754 استفاده کند. از آنجا که جمعِ اعداد ممیز شناور خاصیتِ شرکت‌پذیریِ کامل ندارد، ترتیبِ جمعِ مقادیر باید ثابت و مشخص باشد (برای مثال، جمعِ متوالی به همان ترتیبی که وزن‌ها از SRAM خوانده می‌شوند) تا نتیجه‌ی محاسبات در اجراهای مختلف یکسان و قابلِ بازتولید باشد. پیشنهاد می‌شود سیگنالی مانند last_input برای اعلامِ آخرین ورودیِ هر نورون، و سیگنالی مانند valid_out برای نشان دادنِ آماده بودنِ خروجیِ نهاییِ Hj در این ماژول در نظر گرفته شود.

ثبات‌های لایه دوم

برای لایه‌ی دوم، از آنجا که تعداد ورودی‌ها به‌طور قابل‌توجهی کمتر از لایه‌ی اول است (۸ ورودی به‌جای ۷۸۴ ورودی)، یک رجیستر فایل شامل ۱۰ رجیستر fp16 برای این لایه در نظر گرفته می‌شود. در ابتدای محاسبات لایه‌ی دوم، هشت خروجیِ پس از ReLU لایه‌ی اول در این ۱۰ رجیستر نوشته می‌شوند (دو رجیستر باقی‌مانده بلااستفاده می‌مانند) و به‌عنوان ورودیِ ضرب‌کننده، به ازای محاسبه‌ی هر یک از ۱۰ نورون لایه‌ی دوم، به‌طور مکرر خوانده و با وزن‌های مربوط به همان نورون ضرب می‌شوند.

پس از آنکه جمع‌کننده مقدار خروجیِ (Hj) یکی از نورون‌های لایه‌ی دوم را محاسبه کرد، این مقدار در همان رجیستر فایل و در جایگاه متناظر با آن نورون نوشته می‌شود؛ به این ترتیب همان ۱۰ رجیستر که ابتدا برای نگه‌داری ورودی‌های ضرب‌کننده استفاده شده بودند، مجدداً برای ذخیره‌ی خروجی نهایی هر نورون استفاده

می‌شوند. پس از آنکه مقدار هر ۱۰ نورون محاسبه و در رجیسترها نوشته شد، تمامی ۱۰ مقدار به‌صورت هم‌زمان (parallel) در خروجی ماژول قرار می‌گیرند و سیگنال done فعال می‌شود تا اتمام کامل محاسبات شبکه را اعلام کند.

سایر نکات

- ☐ همراه بودن تصاویر datapath , fsm در کنار کد ها الزامیست.
- ☐ مجاز به استفاده از یک adder و یک multiplier برای انجام عملیات ممیز شناور هستید.
- ☐ توجه کنید که تعداد سیکل کلاک مصرفی برای حل مسئله بخشی از نمره ی تمرین را در بر دارد.
- ☐ فایل های وزن ها و عکس های ورودی همراه کد های تولید کننده آنها در اختیار شما قرار خواهد گرفت، پیکربندی sram بر عهده ی خودتان خواهد بود.